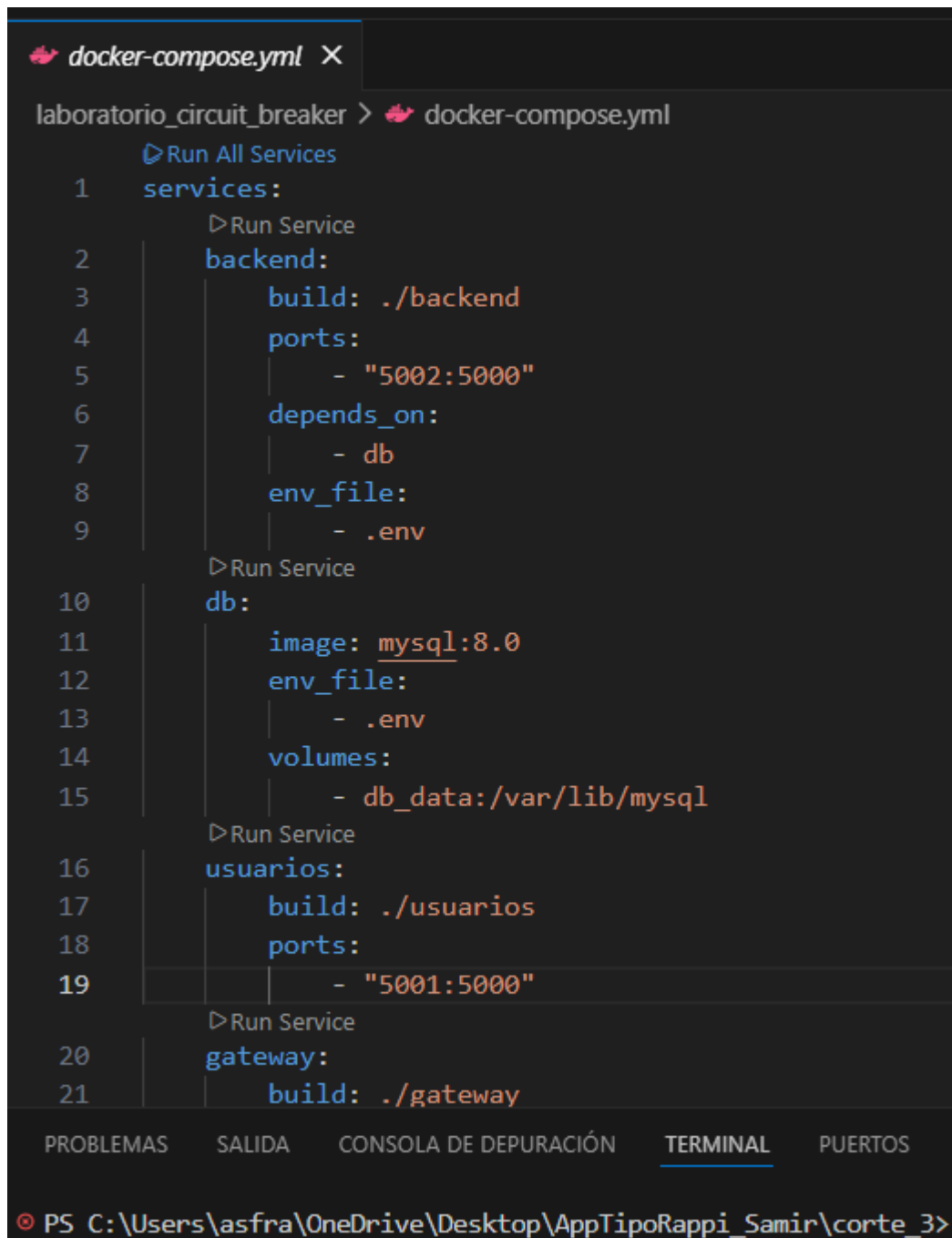


Configuración de archivo docker-compose.yml

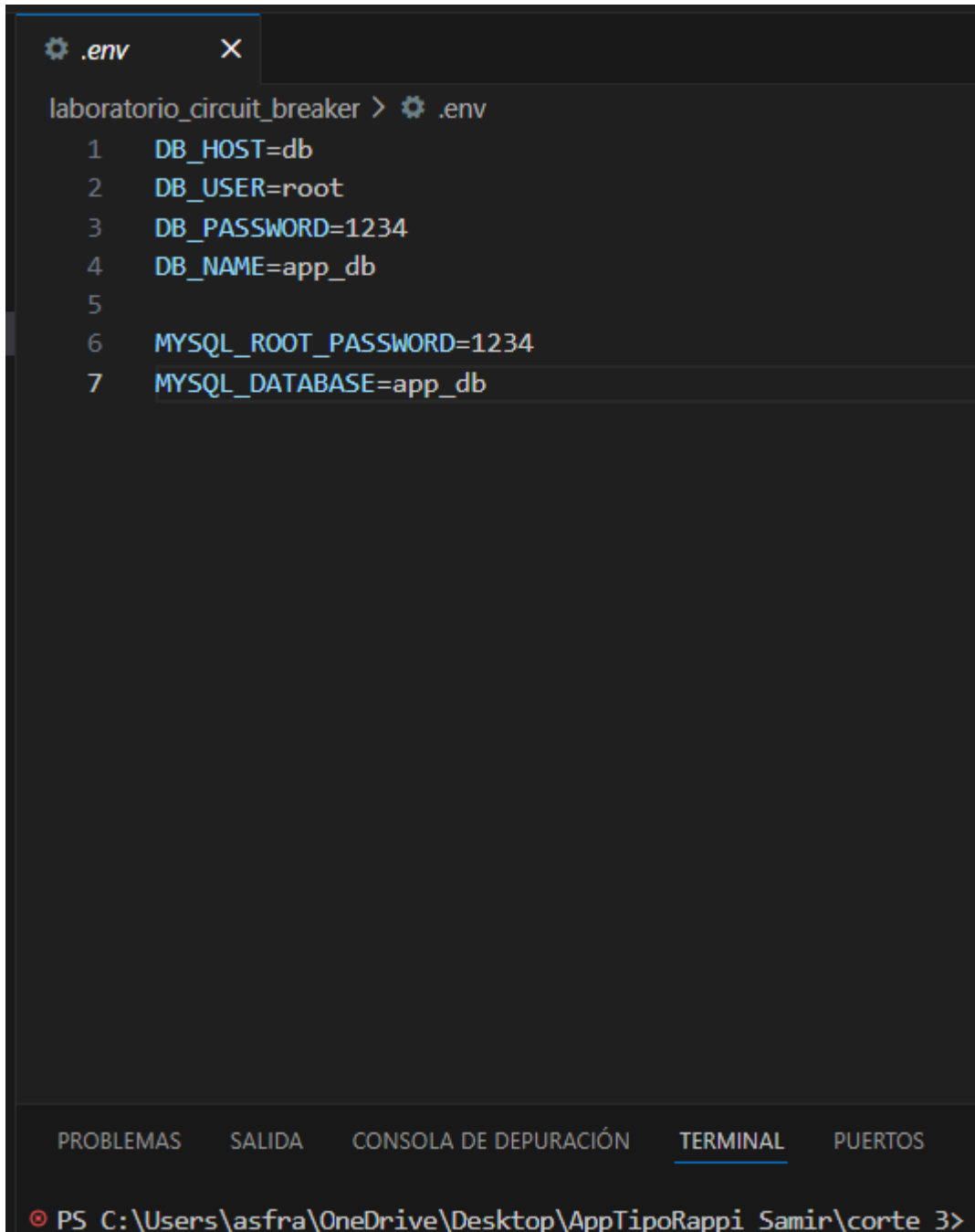
Aquí el código que para que todos los microservicios funcionen juntos. En este archivo se integra el Backend, el Gateway y el servicio de Usuarios con una base de datos MySQL. También se vincula el archivo .env, que es donde se guardan las contraseñas y variables importantes de forma segura para que no estén expuestas directamente en el código.



```
laboratorio_circuit_breaker > docker-compose.yml
  Run All Services
1  services:
    Run Service
2    backend:
3      build: ./backend
4      ports:
5        - "5002:5000"
6      depends_on:
7        - db
8      env_file:
9        - .env
    Run Service
10   db:
11     image: mysql:8.0
12     env_file:
13       - .env
14     volumes:
15       - db_data:/var/lib/mysql
    Run Service
16   usuarios:
17     build: ./usuarios
18     ports:
19       - "5001:5000"
    Run Service
20   gateway:
21     build: ./gateway

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3>
```

En esta sección se muestra el archivo .env, utilizado para gestionar de forma centralizada los datos sensibles del proyecto. Aquí se definieron las credenciales de la base de datos y los parámetros de conexión que consumen los microservicios. Este método es fundamental para evitar colocar contraseñas directamente en el código, permitiendo que el sistema sea más seguro y fácil de configurar al momento de levantar los contenedores.



The image shows a screenshot of a Visual Studio Code editor window. The active file is named `.env`, as indicated by the tab at the top. The editor content shows the following lines of code:

```
laboratorio_circuit_breaker > .env
1  DB_HOST=db
2  DB_USER=root
3  DB_PASSWORD=1234
4  DB_NAME=app_db
5
6  MYSQL_ROOT_PASSWORD=1234
7  MYSQL_DATABASE=app_db
```

At the bottom of the editor, there is a panel with tabs for 'PROBLEMAS', 'SALIDA', 'CONSOLA DE DEPURACIÓN', 'TERMINAL', and 'PUERTOS'. The 'TERMINAL' tab is currently selected, showing a PowerShell prompt:

```
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3>
```

Olvide que se debe abrir el Docker para iniciar con el docker compose up --build

```
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3> cd laboratorio_circuit_breaker
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker> docker compose up --build
unable to get image 'mysql:8.0': failed to connect to the docker API at npipe:///pipe/dockerDesktopLinuxEngine; check if the path is correct
and if the daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker> docker compose up --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.94kB 0.1s done
#1 DONE 0.1s
```

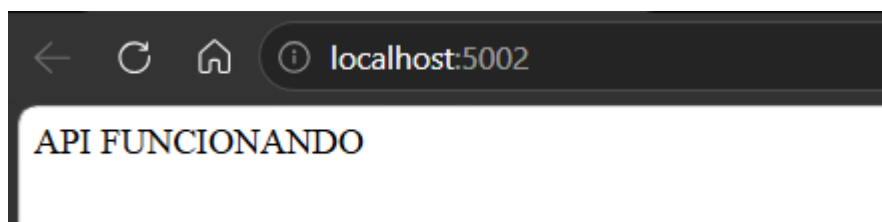
En esta sección se muestra el proceso completo de puesta en marcha del sistema.

Primero, Docker compila las imágenes de cada microservicio instalando las dependencias necesarias.

Finalmente, se visualizan los logs de ejecución en tiempo real, donde se confirma que la base de datos MySQL se inicializó correctamente y que los microservicios (Gateway, Backend y Usuarios) están activos y a la espera de peticiones. Esta fase es crucial para verificar que la comunicación interna entre los servicios se estableció sin errores.

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker> docker compose up --build
#28 DONE 0.0s
[+] up 9/9
✓ Image laboratorio_circuit_breaker-gateway      Built      114.2s
✓ Image laboratorio_circuit_breaker-backend      Built      114.2s
✓ Image laboratorio_circuit_breaker-usuarios      Built      114.2s
✓ Network laboratorio_circuit_breaker_default     Created     0.0s
✓ Volume laboratorio_circuit_breaker_db_data      Created     0.0s
✓ Container laboratorio_circuit_breaker-db-1      Created     0.2s
✓ Container laboratorio_circuit_breaker-usuarios-1 Created     0.2s
✓ Container laboratorio_circuit_breaker-backend-1 Created     0.1s
✓ Container laboratorio_circuit_breaker-gateway-1 Created     0.1s
Attaching to backend-1, db-1, gateway-1, usuarios-1
db-1 | 2026-05-11 00:46:31+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
db-1 | 2026-05-11 00:46:31+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
db-1 | 2026-05-11 00:46:31+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
```

☐	laboratorio_circuit_br latest	c6a51e34d32c	3 minutes ago	1.59 GB			
☐	laboratorio_circuit_br latest	fffc2f12a7e7	3 minutes ago	1.6 GB			
☐	laboratorio_circuit_br latest	0d477d54912a	3 minutes ago	1.8 GB			



```
[+] up 4/4
✓ Container laboratorio_circuit_breaker-usuarios-1 Running 0.0s
✓ Container laboratorio_circuit_breaker-db-1 Running 0.0s
✓ Container laboratorio_circuit_breaker-backend-1 Running 0.0s
✓ Container laboratorio_circuit_breaker-gateway-1 Running 0.0s
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker>
```

En esta captura se utiliza el comando `docker ps` para validar que los cuatro servicios se están ejecutando correctamente. Se puede observar que el Gateway, el Backend, la base de datos MySQL y el microservicio de Usuarios están activos (Status: Up). También se confirma el mapeo de puertos: el Gateway responde en el puerto 5000, mientras que Usuarios y Backend lo hacen en el 5001 y 5002

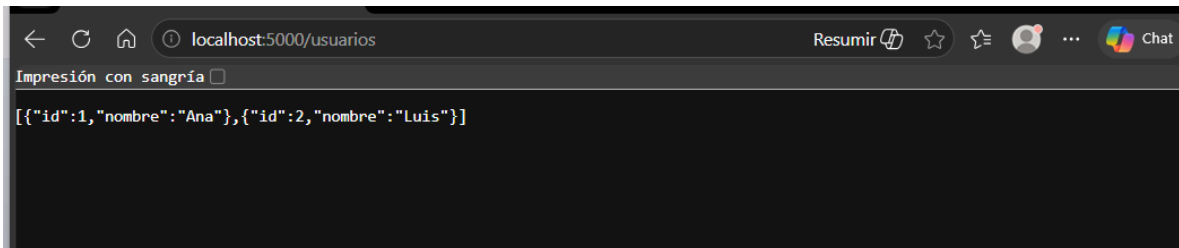
```
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
c3904284bcd   laboratorio_circuit_breaker-gateway  "python app.py"         7 minutes ago Up 7 minutes  0.0.0.0:5000->5000/tcp
, [::]:5000->5000/tcp laboratorio_circuit_breaker-gateway-1
77dac03fdacf   laboratorio_circuit_breaker-backend  "python app.py"         7 minutes ago Up 7 minutes  0.0.0.0:5002->5000/tcp
, [::]:5002->5000/tcp laboratorio_circuit_breaker-backend-1
a97ea764d76d   mysql:8.0                          "docker-entrypoint.s..." 7 minutes ago Up 7 minutes  3306/tcp, 33060/tcp
laboratorio_circuit_breaker-db-1
1c101a3499d5   laboratorio_circuit_breaker-usuarios "python app.py"         7 minutes ago Up 7 minutes  0.0.0.0:5001->5000/tcp
, [::]:5001->5000/tcp laboratorio_circuit_breaker-usuarios-1
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker>
```

Lin. 7, col. 22 Espacios: 4

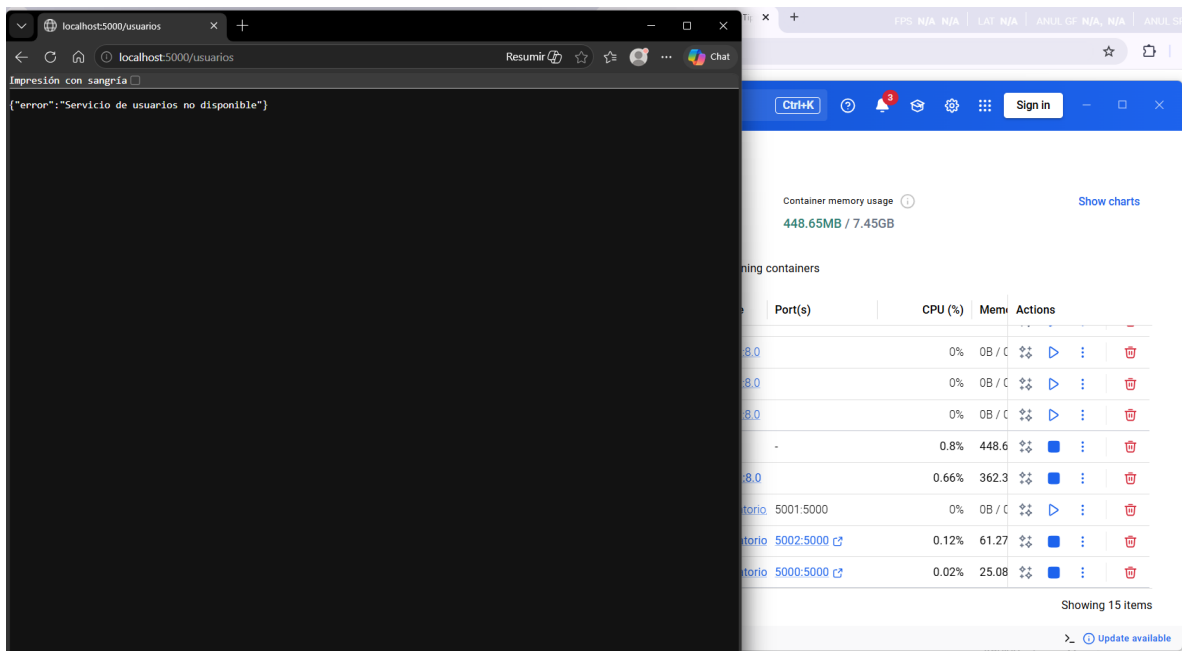
En esta captura se utiliza el comando `docker-compose logs -f usuarios` para supervisar la actividad interna del contenedor de usuarios. Se confirma que el servidor Flask ha iniciado correctamente y está a la escucha de peticiones.

```
PS C:\Users\asfra\OneDrive\Desktop\AppTipoRappi_Samir\corte_3\laboratorio_circuit_breaker> docker-compose logs -f usuarios
usuarios-1 | * Serving Flask app 'app'
usuarios-1 | * Debug mode: off
usuarios-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
usuarios-1 | * Running on all addresses (0.0.0.0)
usuarios-1 | * Running on http://127.0.0.1:5000
usuarios-1 | * Running on http://172.19.0.3:5000
usuarios-1 | Press CTRL+C to quit
usuarios-1 | 172.19.0.1 - - [11/May/2026 00:50:29] "GET / HTTP/1.1" 404 -
usuarios-1 | 172.19.0.1 - - [11/May/2026 00:50:29] "GET /favicon.ico HTTP/1.1" 404 -
```

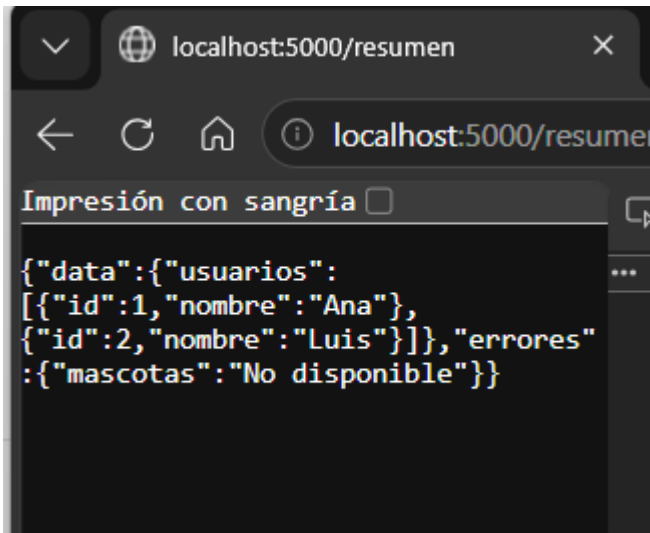
En esta captura se demuestra el funcionamiento integral de la arquitectura. Al consultar la ruta /usuarios a través del puerto 5000, el API Gateway redirige la petición al microservicio correspondiente, el cual obtiene la información desde la base de datos MySQL. El resultado es una respuesta en formato JSON con la lista de usuarios registrados.



En esta prueba se detuvo intencionalmente el microservicio de Usuarios para verificar la capacidad de respuesta del sistema ante errores. Como se observa en la captura, el Gateway identifica la caída del servicio y, en lugar de generar un error crítico de conexión, devuelve un mensaje controlado indicando que el servicio no está disponible.

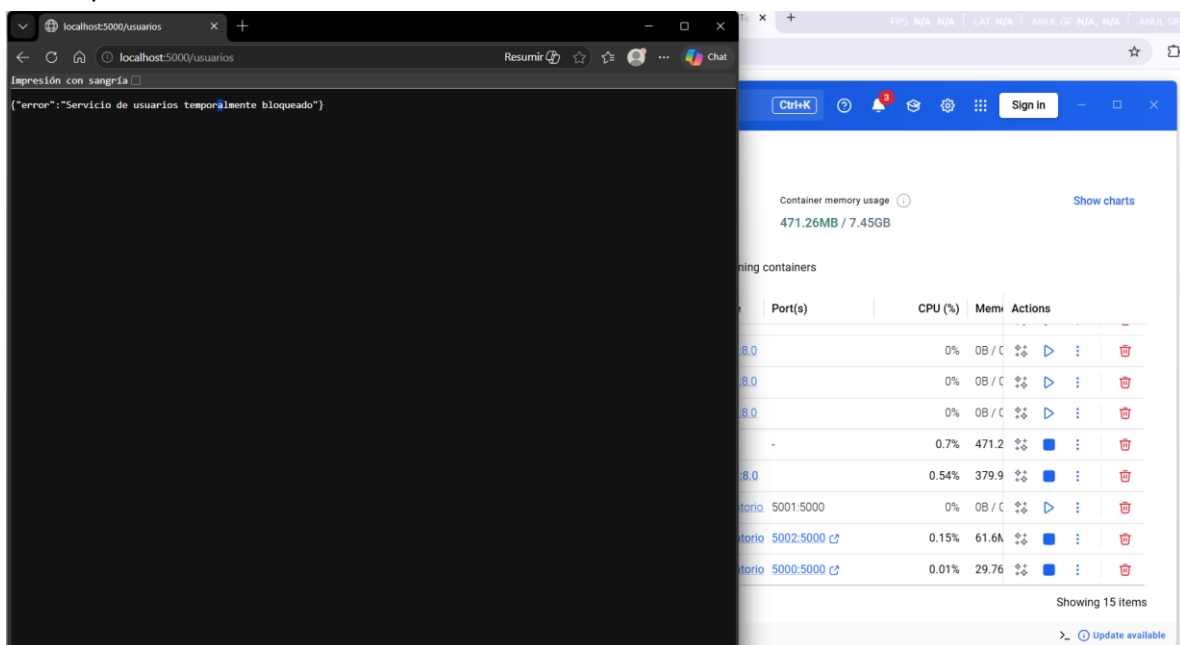


En esta captura se observa una respuesta del Gateway que combina datos exitosos con el manejo de una falla. Mientras que el microservicio de Usuarios responde correctamente devolviendo la lista de nombres desde la base de datos, el sistema detecta que el servicio de Mascotas no está disponible. En lugar de fallar por completo



```
Impresión con sangría ☐
{"data":{"usuarios":
[{"id":1,"nombre":"Ana"},
{"id":2,"nombre":"Luis"}]}, "errores"
:{"mascotas":"No disponible"}}
```

Una vez que el circuito está abierto, el Gateway intercepta cualquier petición entrante y responde de manera inmediata con un error de "Servicio bloqueado". En esta captura se demuestra que el sistema ya no intenta realizar el request al microservicio, cumpliendo con el objetivo de detener llamadas innecesarias mientras el servicio no esté operativo.



The left screenshot shows a web browser at localhost:5000/usuarios displaying the message: `("error":"Servicio de usuarios temporalmente bloqueado")`.

The right screenshot shows a Docker container management interface. At the top, it displays 'Container memory usage' as 471.26MB / 7.45GB. Below this is a table of running containers:

Port(s)	CPU (%)	Memi	Actions
8.0	0%	0B / C	[Icons]
8.0	0%	0B / C	[Icons]
8.0	0%	0B / C	[Icons]
-	0.7%	471.2	[Icons]
8.0	0.54%	379.9	[Icons]
torio 5001:5000	0%	0B / C	[Icons]
torio 5002:5000	0.15%	61.6M	[Icons]
torio 5000:5000	0.01%	29.76	[Icons]

Showing 15 items

